

Lecture 3 — The Execution Model

Jeff Zarnett
jzarnett@uwaterloo.ca

Department of Electrical and Computer Engineering
University of Waterloo

September 5, 2026

So You Want to Run a C Program





First: compiling, linking, loading. Only then, can a process run.

C programs are compiled: the compiler turns the source code into a binary.

Most programs we write in this course are so short they fit in one file.

Real programs are much bigger and spread across many files.

Need a way for `A ( )` in one file to call function `B ( )` in another file, where those two files get compiled into separate units.

Strategy: leave a note!

“When we figure out where `B ( )` is, put its address here.”

ECE 252: use the pthread library and the standard C library.

They are not copy-pasted into the source.

Compiler leaves slightly different kind of note:

“At load time, figure out the address of `write()`, and put its address here.”

(You can copy-paste library code in. That's **static linking**. Not recommended.)



Building the connections between the different parts of the program, is the post-compilation step called **linking**.

Bring the different parts of the program together to create a cohesive program.
Resolve the notes earlier.

Compile-time linking resolves the first kind of note.

The second kind of note is only solved when launching the process.

Example of type 1: gcc says undefined reference to function `do_tihng()` when clearly you meant it to be `do_thing()`.

If the linker cannot resolve all notes, no valid executable produced.

To run an executable, we need to take the program and turn it into a process.

We learned about processes and process control blocks in ECE 252.

`fork ( )` makes an exact copy of a running process (new PCB).

`execve ( )` replaces the executable with a different one.

What does `execve ( )` do?

Step 1: throw away all the things that are to be replaced.

The executable program, memory, other resources (with exceptions).

Survives: process ID, open file descriptors.

Then it's a blank canvas. The OS can load it.

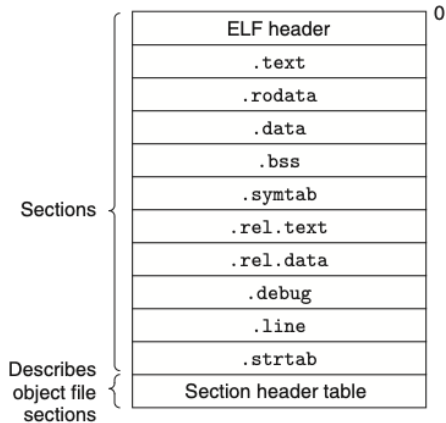
The executable comes with setup instructions (kind of like IKEA furniture).



We'll discuss the **Executable and Linkable Format** (ELF).

Windows, macOS, and other operating systems have their own formats which will differ in some ways.

The different parts of the program that need to be put together are **sections**.



- The **ELF Header**
- The `.text`
- The `.rodata`
- The `.data`
- The `.bss`
- The `.symtab`
- The `.rel.text`
- The `.rel.data`
- The `.debug`
- The `.strtab`
- The **Section header table**



There is much more complexity to how a program is linked against libraries and other dynamically-decided locations than we will have time for right now.

Four sections need to get loaded into memory for the process to get started:
.text, .rodata, .data, .bss.

Each of the four sections to be loaded needs to be put at some memory address.

Exactly where can vary, but they have to be in memory to run.

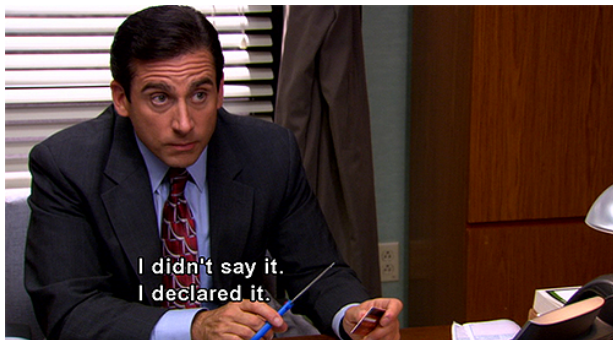
The four are meant to be loaded with permissions; i.e., the `.text` is executable but cannot be written, to prevent accidentally changing the executable code.

We also need to put the stack and heap into memory as well.

These start out as just regions of memory that contain whatever garbage was present in RAM, and then they are available as areas for running the process.

Remember that these are not allocated with something like `malloc()`!

From the operating system's point of view, it can simply declare an area of memory to be the heap for a given process.



The process starts with just the one thread running, but the process can ask for new threads to be created.

That happens after the process has started running, of course, and will require a new stack for each new thread created.

The basic idea is the same, though, it just happens later.

And once we've got things loaded, it's possible to call the process ready to run!

Execution can begin when the scheduler chooses that process.

Execution actually starts slightly before `main ( )` with some setup work (and there's a corresponding cleanup after the end of `main ( )`).

Calling Conventions



The program counter register points to the next statement that the CPU will execute, which is necessary but not sufficient to represent the state of execution.

Function calls keep track of the path in the program.

When `compute ( )` calls `load ( )`, we use the stack to track the execution state.

We want the `load ( )` function to successfully return the result back to the caller.

Knowing the difference between stack and heap means that you can manage memory correctly and avoid memory leaks, crashes, and other bad outcomes.

Variables local to a function appear in the stack (or in registers).

Things allocated dynamically with `malloc()` are in the heap.

The stack for program execution is the same as the one for these variables.

When `load ( )` is called, a new **stack frame** must be created.

In addition to the local storage for that function, the stack frame is used for passing arguments, storing the return information, and is used to save registers.

Growth direction may vary.

Function return: pop something off the stack.

Function arguments can be passed via the stack or registers.

32-bit x86 used the stack.

In the 64-bit x86 architecture, registers enter the picture. They're faster!

There's rules about the order of register filling.

If function $A()$ is calling $B()$, the registers for $A()$ are likely being used for something.

There is further complexity about which registers are saved by the caller and which are saved by the callee.



Start the new function: update the program counter.

The assembly instruction to go to that destination not only transfers control, but it also writes down the return location.

The return information includes where execution goes back to, when the current function ends.

Imagine the function call looks like `int b = get_open_balance();`.

`return 0;` at the end of the function sends 0 back to the caller.
Send it back via a register if possible.

Execution resumes in the caller.



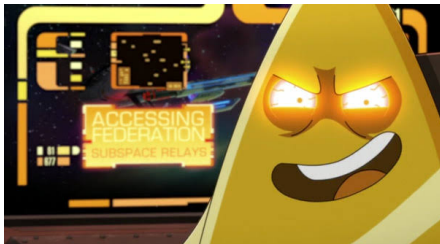
Given that the return location is just sitting there in the stack frame, it is vulnerable to malicious actors.

Attacker goal: redirect execution via changing that address.

One way that a malicious actor can do this is a **stack smashing** attack.

The goal here is to write too much data into a stack-allocated buffer.

The program must be vulnerable.



- 1 Address Space Layout Randomization
- 2 Stack Canary
- 3 NX Processor Feature

These mitigations specifically stop the redirection of control flow to the attacker's desired location.

It is still possible to use a stack-smashing attack to crash the process by changing the return address to something invalid.

This can be used to execute a denial-of-service attack, though, by repeatedly crashing the process.

In ECE 252, we discussed interrupts, starting with sending signals.

The canonical first example introduces a handler for SIGINT that changes a boolean `quit` to be true.

Some signals like SIGINT are **asynchronous**.

As previously discussed, there are also synchronous signals.

Recall that the signal is delivered by the operating system to the process.

Signal handlers need a stack frame.

The kernel must push one onto the stack.

There is special unwinding code at the end of the interrupt.

Many interrupts arise from hardware and are handled by the kernel with its own interrupt handling code.

The kernel may choose, if appropriate, to use signals as the way of notifying the user process of what happened.

Example: close the lid of the laptop.

Even SIGINT has some hardware interrupt behind it!

Hardware interrupt handlers do not end with normal return statements, but often something like `rti`, `rte`, or `iret`.

Why?

The `return` statement does not work; its semantics (as discussed earlier) terminate the current function and return control to the calling function.

An interrupt handler should instead send execution back to where it was before the interrupt took place. There is no calling function to go back to.

We have a reasonably good idea of how to run a program and how it executes.

That only works, however, when the operating system itself is running.

Let us consider a day in the life of the operating system next.